

ModelGate

Platform Audit Kualitas Dataset Computer Vision

UAS — Web Service & Pemrograman Berbasis Platform

Agrieby Chaniago — S1 Informatika, Semester 6

Agenda

1. Latar Belakang & Rumusan Masalah
2. Metode: Arsitektur & Teknologi
3. Fitur-Fitur Pengembangan UAS
4. Cara Kerja Sistem (Alur End-to-End)
5. Demo Langsung
6. Kesimpulan & Pengembangan Selanjutnya

Latar Belakang

Kualitas dataset adalah **penentu utama performa model CV** — bukan arsitektur modelnya.

Masalah yang sering luput dari pengecekan manual:

- File gambar rusak, tapi tetap terbaca sistem
- Gambar duplikat → model overfitting
- Distribusi kelas timpang → model bias
- Resolusi tidak konsisten → augmentasi terganggu

Tanpa audit → masalah baru ketahuan setelah **berjam-jam compute time terbuang**.

Solusi: ModelGate

ModelGate = **penjaga gerbang**, bukan tools training model.

Pengumpulan Data → [MODELGATE AUDIT] → Preprocessing → Training

Upload dataset ZIP → 5 analyzer otomatis → **Health Score** → Laporan PDF

Rumusan Masalah untuk UAS

Project sudah berjalan sebagai microservices dasar (Fase 1–5).

Untuk UAS, dikembangkan fitur yang relevan ke **dua** mata kuliah sekaligus:

Web Service

- Autentikasi JWT + 3 tier (Free/Pro/Max)
- API Key + CLI (akses terprogram)
- Frontend React + WebSocket real-time
- Rate limiting di API Gateway

Pemrograman Berbasis Platform

- Observability (Prometheus + Grafana)
- Horizontal scaling
- CI/CD (GitHub Actions)

Arsitektur — Microservices

Service	Peran
<code>dataset_service</code>	Upload & manajemen dataset
<code>audit_service</code>	Orkestrasi audit, state machine
<code>analysis_service</code>	5 analyzer gambar (consumer RabbitMQ)
<code>report_service</code>	Health Score & laporan PDF
<code>auth_service</code> (baru)	JWT, API Key, tier

Semua di belakang **Nginx API Gateway** (port 8080) — satu pintu masuk untuk semua klien.

Arsitektur — Infrastruktur

- **RabbitMQ** — proses berat (analisis gambar) berjalan asinkron, tidak blocking HTTP
- **MinIO** — object storage (S3-compatible) untuk gambar
- **PostgreSQL** — satu database, **schema terpisah per service**
 - Tiap service hanya menulis ke schema-nya sendiri
 - Baca lintas service → pola *read-only mirror*

Prinsip: **bounded context** dijaga konsisten di semua fitur baru.

Fitur 1 — JWT Auth + Tier

- `auth_service` baru: register, login, JWT dengan klaim `plan`
- Nginx `auth_request` → validasi token di gateway sebelum diteruskan
- **Nginx hanya autentikasi** ("siapa kamu") — otorisasi ("boleh apa") tetap di tiap service

Tier	Upload	Analyzer	Kuota/hari	PDF
Free	150MB	3 dari 5	3	×
Pro	1GB	5	20	✓
Max	2GB	5	∞	✓

Fitur 2 — API Key + CLI

- API Key eksklusif Pro/Max — untuk **akses terprogram**, bukan manusia di browser
- CLI `mgs` (Python) — upload, audit, monitor progres live, unduh laporan
- Satu command: `mgs run dataset.zip --pdf`

| Skenario: ML Engineer otomatisasi cek kualitas dataset di pipeline mereka sendiri.

Fitur 3 — React Frontend + WebSocket

- Frontend baru: **React + Vite + Tailwind** (Streamlit lama tetap jalan paralel)
- Progres audit **real-time** via WebSocket, bukan polling

Bug ditemukan saat testing: klien yang connect *setelah* audit selesai (dataset kecil) menunggu selamanya — tidak ada replay pesan.

Fix: pola *snapshot-then-subscribe* — kirim status terkini begitu klien connect, baru lanjut stream event baru.

Fitur 4 — Rate Limiting

- Nginx: batas berbasis IP — endpoint login lebih ketat (brute-force protection), endpoint lain lebih longgar
- Percobaan rate-limit per-tier langsung di Nginx **dibatalkan** — kendala teknis urutan eksekusi modul (`auth_request` vs `limit_req`)
- Solusi: kuota harian per-tier dipindah ke level aplikasi (`audit_service`)

Fitur 5 — Observability

- **Prometheus** — metrik otomatis dari 5 backend service (request rate, latency)
- Plugin Prometheus bawaan **RabbitMQ** — kedalaman antrian & consumer count, tanpa kode tambahan
- **Grafana** — dashboard visual, auto-provisioned

Fitur 6 — Horizontal Scaling

`analysis_service` sudah terhubung ke RabbitMQ sebagai consumer sejak awal.

Ternyata arsitektur sudah mendukung multiple consumer paralel, otomatis dibagi rata oleh RabbitMQ — cukup hapus satu batasan port:

```
docker compose up -d --scale analysis_service=3
```

Nol kode load-balancing baru ditulis.

Fitur 7 — CI/CD

GitHub Actions: build + push image Docker tiap service ke **GitHub Container Registry**, otomatis setiap push ke `main` .

Alur Kerja End-to-End

1. Login → JWT dengan klaim `plan`
2. Upload → Nginx → `auth_request` → `dataset_service` (cek batas sesuai plan, hash SHA-256, simpan ke MinIO)
3. Buat audit → `audit_service` tentukan analyzer **di server** (bukan dari input klien)
4. Job → RabbitMQ → `analysis_service` jalankan analyzer satu-satu
5. Tiap hasil analyzer → broadcast WebSocket (browser & CLI, channel sama)
6. Selesai → `report_service` hitung Health Score

Health Score

$$\text{Score} = 0.30 \times I + 0.25 \times U + 0.25 \times D + 0.20 \times Q$$

- **I**ntegrity = $1 - \text{corruption rate}$
- **U**niqueness = uniqueness rate (pHash)
- **D**istribution = $1 - \text{gini coefficient}$
- **Q**uality = gambar dalam $\pm 1\sigma$ resolusi median

Grade: A (≥ 0.80) — B — C — D (< 0.40)

Bug yang Ditemukan Saat Testing

Race condition di `audit_service` (bukan kode baru — bug lama):

Publish job ke RabbitMQ terjadi **sebelum** commit status "queued" → consumer bisa baca status stale → job macet permanen.

Fix: commit status dulu, baru publish.

WebSocket tanpa replay (dijelaskan di slide Fitur 3).

| Bukti proses testing menyeluruh, bukan asumsi kode sudah benar dari awal.

Demo Langsung

Autentikasi & Tier · API Key & CLI · Rate Limiting · Horizontal Scaling · Observability

Kesimpulan — Web Service

- REST API dengan API Gateway (Nginx)
- Dua skema autentikasi: JWT (manusia) + API Key (programmatic)
- Komunikasi real-time via WebSocket
- Rate limiting di gateway

Kesimpulan — Pemrograman Berbasis Platform

- Containerization dengan Docker & Docker Compose
- Message queue asinkron (RabbitMQ)
- Observability (Prometheus + Grafana)
- Horizontal scaling
- Automasi CI/CD (GitHub Actions)

Pembelajaran

Nilai terbesar bukan cuma implementasi fitur baru — tapi proses **debugging dan testing menyeluruh**.

Bug race condition dan WebSocket ditemukan karena sistem benar-benar dijalankan dan diuji, bukan diasumsikan benar dari awal.

Pengembangan Selanjutnya

- Orkestrasi dengan **Kubernetes** (sengaja di luar scope UAS)
- Reconnect otomatis untuk WebSocket yang terputus
- Dashboard Grafana dengan alerting

Terima Kasih

Repository: github.com/agriby-chaniago/MGS